
Assignment 2 for APML24/25@UU

Irvan Ilahibaks, Iván González, Fabian Linsenmeier, Cas Bouwman
Group 5

Abstract

In this study, we apply Predictive Process Monitoring (PPM) to measures performed on a patient in a hospital after a diagnosis of sepsis. Based on patient characteristics and events in the hospital (e.g., IV, ER, CRP, leucocyte measurements), we try to predict the remaining time of the patient in the hospital before the eventual release. The results show the promise of PPM techniques in processing clinical event data.

1 Introduction

The data in *sepsis.csv* consists of an event log that contains the case ID variable which serves as a unique identifier for each patient. The goal is to predict the remaining hospital time of the patient at each event.

Section 3.1 explains the data preprocessing and data transformation. The event log is trace-encoded and split temporally into training set and test set based on the timestamp of the event. We train several algorithms on the resulting dataset in section 3.2: Regression Tree, Random Forest, kNN and MLP. Bucketing is explained in section 3.3, where we also apply it to the presented task. Lastly, we try out alternative preprocessing and encoding methods in section 3.4 (feature selection, elapsed time, autoencoder).

2 Data exploration

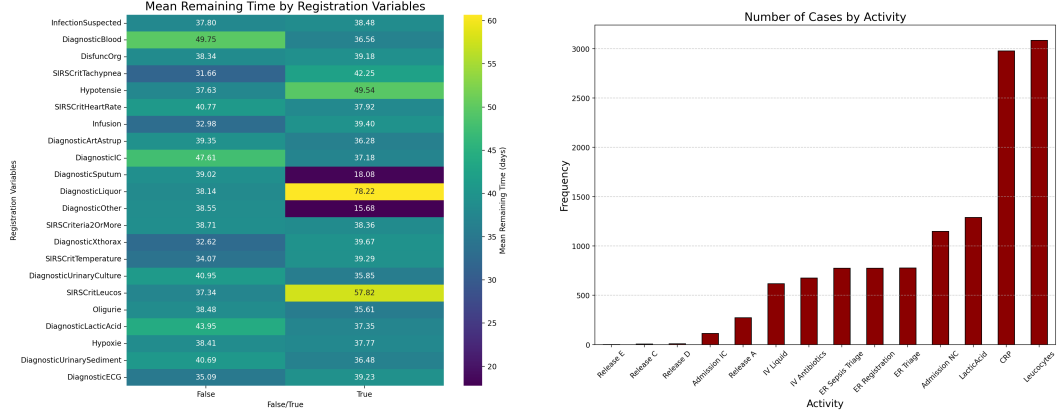
The dataset consists of many categorical variables which are illustrated in this section. When patients are registered at the hospital, they undergo first diagnostics. Figure 1a displays the average remaining time based on these registration variables. While most variables do not show significant impact, diagnosed liquor and SIRSCritLeucos seem to correlate significantly with longer time in the hospital. On the other hand, e.g. diagnosed sputum seems to correlate with a relatively quick release from the hospital. After registration, consecutive activities are implemented to help the patient recover from their sepsis. Figure 1b gives an overview over the activities. Most frequently performed are CRP and leucocyte measurements.

3 Prediction of remaining time

3.1 Preprocessing of the Data

With a dataset such as this, it is important to preprocess the data as it contains, either a lot of empty cells or cells that do not contain a number. Some of these may just be cell that are filled or left blank in error, but for a lot of instance they could hold useful feature data.

For this purpose it requires us to look at the application of our data set. We are looking at the events that happen to a patient over the course of their stay in a hospital and predicting the time they have left in the hospital. We want each instance of an event to contain some information about their history in the hospital. For this purpose we implemented aggregated state encoding, as done in the source paper [1]. Now each instance contains how many times a patient has undergone a certain activity at



(a) Average hospital time for the given registration variables. Variable values correlated with longer hospital time are displayed in yellow. Variable values correlated with shorter hospital time are colored in darkblue.

(b) Bar plot showing the frequencies of the different activities in the event log.

Figure 1: Data exploration: Registration variables and activity frequencies.

that time. Some of these events also have results reported for the activity in that event. These results are only reported for that event and nowhere else. Similar to the aggregated encoding we want to see at any time what the results were for some test. However where aggregated state encoding computes the sum we are only interested in the most recent test value. So we repeat the reported value until it is updated by another test.

For most cases there are no test values reported at the beginning. This means that it has no value to repeat. Instead these are replaced with a default value. This default value is calculated by taking the median of all reported values for the test that a patient undergoes.

In addition the features "Diagnose" is run through a hashing function to ensure that it is a numeric type.

Finally we eliminated some events as they are prone to error. As said in the source paper this type of activity is often given a wrong timestamp which might influence the results.[1]

For training and evaluating models we require the dataset to be split into a training set and a test set. The data set is split according to some timestamp $T_{threshold}$. All cases that start prior to this timestamp will belong to the training set and all cases that start after this timestamp will belong to the test set. Note that this means that the timestamps of some events in the training can overlap with an event in the test set. This is a consequence of keeping the entirety of case in a set. This is done as such to hopefully let the models learn the context of an entire case. $T_{threshold}$ is chosen algorithmically. We want to choose a threshold that results in a good training and test split. Therefore it is chosen such that the data set is split 80/20, (e.g. 80% training and 20% test). This is done by sorting the cases by their starting time and setting $T_{threshold}$ to be the starting time of the case that splits the sorted cases 80/20.

3.2 Regression models

Now we will talk about four regression models that we use to see which of the four fits best for the data provided. These four regression models are: Decision Tree Regressor, Random Forest Regressor, K-Nearest Neighbors Regressor and MLP Regressor. We will also explain the use of Random State 42 and the use of Grid Search CV.

Decision Tree Regressor

Lets start with the `DecisionTreeRegressor` (code), it is a type of machine learning model that used to predict numbers. It works by making decisions based on the data's features, that looks like a flowchart. The model splits the data into smaller parts based on the different features, creating a tree

like structure. Each branch of the tree represents a decision and each leaf shows the predicted value. They can handle complex data by finding patterns that are not just straight lines which makes them versatile for different types of data. Though we gotta keep in mind that decision trees can sometimes learn the training data too well. This includes noise, which makes them less accurate on new data. This is known as overfitting, which a possible fix can be limiting the trees depth.

Random Forest Regressor

The `RandomForestRegressor` (code) is a group of decision trees working together to make better predictions. It builds many trees and combines their results to get a more accurate and stable prediction. Each tree is trained on a random part of the data and uses a random selection of features, which helps make the overall model stronger and less likely to overfit. Using multiple trees improves the models performance because it averages out the errors from individual trees. It is also more reliable than noisy data and outliers when compared to a single decision tree. That makes it a strong choice for many regression tasks.

Random state 42

In our code, we set the `random_state` parameter to 42 for both `DecisionTreeRegressor` and `RandomForestRegressor`. This helps ensure that the results are the same every time we run the model by controlling the random parts of the training process.

The number 42 is commonly used in programming and data science as a default value. It's a fun reference to Douglas Adams' book *The Hitchhiker's Guide to the Galaxy*, where 42 is humorously presented as the "answer to the ultimate question of life, the universe and everything". Using a fixed random state makes sure that running the model multiple times gives the same results. This consistency is important for testing and comparing different models fairly.

K-Nearest Neighbours Regressor

Next in line is `KNeighborsRegressor` is a simple model that makes predictions based on the closes data points in the feature space. To predict a new value, it looks at the 'k' nearest neighbours (similar data points). Afterwards it avarages their target values. The closeness is usually measured using distances like the Euclidean Distance. KNN is easy to understand and implement because it does not require a separte training phase. It can adapt to different data patterns by changing the number of neighbours (K). This allows for flexibility in how it makes prediction. However, KNN can be slow when working with large datasets, because it needs to calculate distances for all points every time it makes a prediction. This makes it less suited for very large datasets unless it gets optimized first.

MLP Regressor

Finally we have the `MLPRegressor`, MLP stands for Multi-Layer Perceptron Regressor. This regressor is a type of neural network. Unlike the other regression models that have been discussed, the MLP Regressor can capture more complex patterns in the data by using multiple layers of neurons. It consists of an input layer, one or more hidden layers and an output layer. Each layer is made up of neurons that process information and pass it to the next layer. The connections between neurons have weights that are adjusted during training to improve the models predictions. One of the main advantages of the MLP Regressor is its ability to model non-linear relationships between the input features and the target variable. This makes it very powerful for tasks where the relationship between variables is not straightforward. Additionally, by adding more hidden layers and neurons, the MLP Regressor can learn more detailed patterns in the data. However, there are also some challenges. It requires more data and computational power compared to simpler models like Decision Trees or k-Nearest Neighbors. Training a neural network involves finding the right weights for all the connections, which can be time-consuming. Moreover, if the network is too complex it might overfit the training data. This will mean it performs well on the data it was trained on but poorly on new unseen data.

3.3 Bucketing

Bucketing is a technique used to group sequences of events based on their prefix length. By dividing data into buckets, we can analyze and model behaviors at varying stages of the sequence, enabling a more focused analysis tailored to the length of observed activity.

Specifically, we divided the dataset into buckets with prefix lengths of 5, 10, and 15. We applied the same preprocessing steps and trained the Random Forest regressors following the same methodology as in the models without bucketing. The performance of each bucket was evaluated using key metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), R^2 . The results are as follows:

$$MAE = 4.32, \quad MSE = 50.37, \quad R^2 = 0.03 \quad \text{Prefix : 5} \quad (1)$$

$$MAE = 4.61, \quad MSE = 65.41, \quad R^2 = -0.19 \quad \text{Prefix : 10} \quad (2)$$

$$MAE = 6.71, \quad MSE = 95.39, \quad R^2 = -0.13 \quad \text{Prefix : 15} \quad (3)$$

The results demonstrate that the model’s performance varies depending on the prefix length used for bucketing. The bucket with a prefix length of 5 achieved the best results; in contrast, the performance deteriorates as the prefix length increases. This suggests that shorter prefixes provide a better balance of information and model generalization.

3.4 Alternative Encoding methods

Feature selection: The current feature set contains 27 features. While this is not a particularly large number of features it is always useful to see whether a smaller subset of features is able to encode the same amount of information. The main dataset consist entirely of events. The thing that mainly characterizes a single event is the activity undertaken for that event, with most of the other features being results of those activities. Therefore considering only the activities as features might already be enough to adequately predict the remaining time. For this purpose we only consider the features that encode activity. However we still consider them in their aggregated encoded state. This way a single event still gives us some context on the rest of the patient’s history in the hospital. This reduced feature set performed significantly worse than the complete feature set on the Random Forest. It increases the MSE from 69 to 124. A possible explanation might be that the results of certain activity such as Leucocytes can dictate the next event, (e.g. a Leucocyte score might be so concerning that it causes the doctors to keep the patient longer and vice versa). This means that those values are useful for predicting remaining time in the hospital.

Elapsed time: Another features that might be useful to consider is the amount of time that the patient has already been in the hospital. A hospital can have a policy that a patient cannot stay longer than a certain amount of time. Another possibility might be that every case that the hospital processes is that they are all finished in the same or similar amount of time. In theory in these situations adding the amount of time that a patient has been in the hospital might improve performance. Elapsed time would be directly negatively correlated with the time remaining in the hospital as the sum of the elapsed time and the remaining time at any point should add up to a constant, the total time of stay in the hospital. However in practice this does not seem to improve the performance of the Random Forest. The MSE of this model is 95. This is a significant difference from the previous MSE of 69. It seems that in reality not all hospital cases adhere to the previous described situations. Cases have different staying times which means that while the elapsed time is still negatively correlated with the remaining time the constant that they add up to is different for each case, causing further confusion when predicting remaining time for test data.

Lastly, we implemented an autoencoder to learn the latent features more efficiently. The 27-dimensional input is encoded into a bottleneck layer via an MLP and then decoded again to 27 dimensions to mimic the input. We tried out different sizes of the latent space and found a bottleneck size of 6 to be the most suited for the task. To compare the effect of the feature learning, we repeated the Random Forest Regression with and without prior autoencoding. The results do not show an improvement of the RF because of the Autoencoder: The autoencoder increases the validation MSE from 69 to 96. An explanation lies in the structure of the RF. Since it always chooses first the variable with maximum entropy gain, there is not really a need to use feature learning prior to the RF (other than saving computation time). Rather, the reduced dimensionality after the AE seems to contain less information and ends up performing worse in the Random Forest. We conclude that the AE applies more to situations where dimensionality reduction is needed.

4 Results and Discussion

Encoding	Model	Training MAE	Test MAE	Training MSE	Test MSE	Training R^2	Test R^2
Agg-state and no bucketing	RF	0.27	5.08	0.22	68.8	1.00	-0.10
Agg-state and no bucketing	DT	$6.58 * 10^{-4}$	6.54	$7.07 * 10^{-4}$	99.30	1.00	-0.58
Agg-state and no bucketing	kNN	1.20	5.91	8.18	97.28	0.95	-0.098
Agg-state and no bucketing	MLP	4.10	6.09	33.84	85.83	0.80	-0.37
Agg-state and prefix length 5	RF	1.71	4.32	8.25	50.37	0.86	0.03
Agg-state and prefix length 10	RF	1.80	4.62	9.71	65.41	0.85	0.2
Agg-state and prefix length 15	RF	2.43	6.71	17.16	95.39	0.85	0.13
Agg-state reduced feature set	RF	4.29	6.57	57.85	124.37	0.66	-0.98
Agg-state elapsed time	RF	2.43	6.71	17.16	95.39	0.85	-0.13

Table 1: Performance metrics for regressors with various encodings and bucketing strategies.

Based on the error measures on the test set, the optimal method appears to be the combination of aggregated state encoding, prefix length of 5, and the Random Forest model. This combination achieved the best balance of performance metrics, particularly in terms of Mean Absolute Error (MAE) and Mean Squared Error (MSE), compared to other configurations.

For selecting the model in this context, Mean Absolute Error (MAE) would be a preferred metric. MAE provides a straightforward interpretation by measuring the average magnitude of prediction errors in the same unit as the target variable (hospital stay in days). Additionally, MAE is easy to communicate to healthcare professionals, as it directly indicates how many days, on average, the model's predictions deviate from the actual values

5 Conclusions

The results of the study demonstrated the potential of predictive process monitoring (PPM) techniques for clinical event data but also highlighted some challenges in selecting the optimal preprocessing, encoding, and modeling methods.

During the first step of the study, certain registration variables, such as diagnosed liquor and SIRSCritLeucos, were identified as significant indicators of longer hospital stays, while others, like diagnosed sputum, correlated with shorter stays. This highlights the importance of exploring event data to uncover meaningful patterns that can inform patient care strategies.

The preprocessing steps proved essential in preparing the dataset for analysis, particularly in addressing missing values and ensuring feature consistency. Aggregated state encoding was implemented to provide historical context for each event, while default values were calculated to handle missing test results.

After the preprocessing, four regression models were evaluated for predicting remaining hospital time: Decision Tree, Random Forest, k-Nearest Neighbors, and MLP regressors. Among these, the Random Forest model demonstrated superior performance. Further analysis revealed that shorter prefixes in the bucketing approach yielded better results, with a prefix length of 5 achieving the lowest errors (MAE = 4.32, MSE = 50.37). Longer prefixes, on the other hand, resulted in diminished model performance, suggesting that shorter prefixes provide a more effective balance between information density and generalization. And even though this combination achieved the best performance on test set in terms of error metrics, the result of 4.32 for MAE is still not acceptable in this context. The key issue lies in the scale of the dataset: the mean remaining time in the hospital is 9 days. This means that on average, patients are expected to stay in the hospital for 9 days. An MAE of 4.32 indicates that the model's predictions are off by over 4 days on average.

Thus, while the model performs well in a theoretical sense, the level of error is too large for practical use, especially given the relatively short average length of hospital stays. For more reliable predictions, further improvements in the model or adjustments in the features might be necessary to reduce the error rate to a level that can meaningfully impact healthcare decision-making.

Also, alternative encoding methods were also explored to enhance model performance. While reducing the feature set to only activity-related features simplified the dataset, it significantly worsened predictive accuracy, indicating that non-activity features such as leucocyte results are crucial for accurate predictions. Incorporating elapsed time as an additional feature provided theoretical benefits, as it is inherently correlated with remaining time.

Overall, the study underscores the effectiveness of combining comprehensive preprocessing techniques, thoughtful feature engineering, and machine learning models for predictive tasks in clinical settings. The findings highlight both the promise and the challenges of applying PPM to healthcare, offering insights for future research to further refine these methodologies.

References

- [1] Felix Mannhardt, Daan Blinde, Eindhoven University of Technology, and Blijndier. Analyzing the Trajectories of Patients with Sepsis using Process Mining. Technical report.